Annotations and Labels

2026-04-17

Introduction

A figure that shows the data without any annotation forces the reader to do the interpretive work alone. A figure with too many annotations buries the data under labels, arrows, and ornamental boxes. The middle ground — a few well-placed labels, an arrow that points at the one observation worth singling out, a rich-text title that names the comparison — is what separates publication-quality plots from informative-but-amateurish ones. The `ggplot2` ecosystem provides three families of annotation tools that cover virtually every requirement: `annotate()` for single hand-placed items, `geom_text` / `geom_label` (and the repelling `ggrepel` variants) for data-driven labels, and `ggtext` for rich Markdown and HTML inside titles, axes, and strip text.

Prerequisites

A working knowledge of `ggplot2`'s grammar of graphics, layers, and aesthetic mappings.

Theory

- `annotate()` adds a single geom (text, segment, rectangle, point) at hard-coded coordinates with no aesthetic mapping from the data; this is the right tool for a one-off label or arrow that does not depend on a row of the data frame.
- `geom_text()` and `geom_label()` map labels to rows of the data, drawing one label per row at the corresponding (x, y) coordinates.
- `ggrepel::geom_text_repel()` and `geom_label_repel()` solve the universal overlap problem by iteratively pushing labels away from each other and from the data points, with adjustable force, segment lines, and exclusion regions.
- `ggtext::element_markdown()` parses Markdown and a subset of HTML in axes, legends, titles, and strip text, enabling inline italics, bold, colour, and superscripts without dropping into expression syntax.

Assumptions

No statistical assumptions; the only requirement is that text strings be on the same scale as the surrounding data so labels appear in the right region of the plot.

R Implementation

```r
library(ggplot2); library(ggrepel); library(ggtext)

p <- ggplot(mtcars, aes(wt, mpg)) +
  geom_point() +
  theme_minimal()

# Single annotation
p + annotate("text", x = 3, y = 30, label = "Lightweight, efficient", size = 5)

# Labelled points with repel
```

```r
p + geom_text_repel(aes(label = rownames(mtcars)), size = 3)

# Arrow with annotation
p + annotate("segment", x = 5, xend = 4, y = 25, yend = 20,
             arrow = arrow(length = unit(0.3, "cm"))) +
  annotate("text", x = 5.1, y = 25.5, label = "Outlier")

# Markdown in titles
p + labs(title = "Weight vs <span style='color:#2A9D8F'>*mpg*</span>") +
  theme(plot.title = element_markdown())
```

Output & Results

Four variations on the same scatterplot: one with a hand-placed text annotation, one with each car labelled by name and laid out by `ggrepel` so labels do not collide, one with an arrow drawing attention to a single point, and one with a Markdown-formatted title. The point geom remains identical across all four; only the annotation layer changes.

Interpretation

A reporting sentence: "We labelled the four cars with the highest fuel efficiency using `ggrepel::geom_text_repel()` (force = 1, max.overlaps = 10), and emphasised the outlying Maserati Bora with a hand-placed arrow." When annotations carry interpretive content, describe them in the figure caption so readers do not have to guess what is being highlighted.

Practical Tips

- Prefer `ggrepel::geom_text_repel()` over `geom_text()` whenever labels can overlap; the repelling variants always look more polished and are robust to data updates.
- `annotate()` is the right tool for a single fixed label; using `geom_text()` with a one-row tibble is unnecessarily verbose.
- For a small set of labels with the same style, build a small data frame and map all aesthetics through `geom_text()` rather than stacking many `annotate()` calls.
- Keep annotations away from data marks, legends, and figure margins; if you cannot, increase the plot size before re-styling the labels.
- Use `ggtext::element_markdown()` for inline italics in scientific names, bold for statistical thresholds, and colour for paired emphasis between text and data; it is far cleaner than `expression()`-based plotmath.
- Save the labelled and unlabelled versions of any figure separately; reviewers occasionally request a "clean" version for the supplement.

R Packages Used

`ggplot2` for the base plot system, `ggrepel` for collision-free labels, and `ggtext` for Markdown/HTML rendering inside themes; `cowplot` and `patchwork` complement the workflow when you need to compose annotated figures into multi-panel layouts.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**

- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- ggplot2 grammar and multi-panel layouts